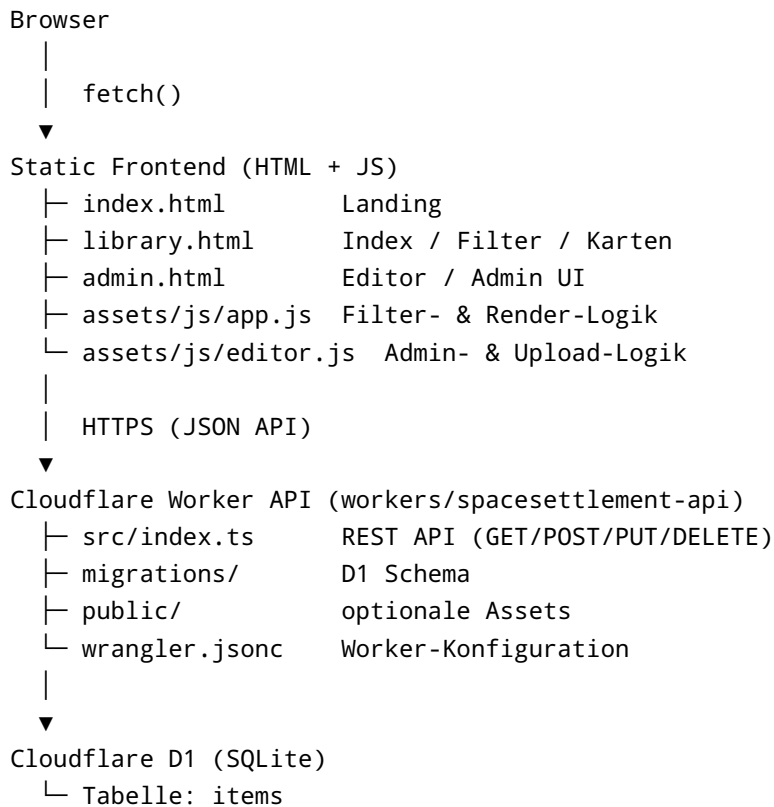


Space Settlement

Architekturübersicht



Datenfluss (vereinfacht)

```
Admin UI → editor.js → POST/PUT /items → Worker → D1
Library UI → app.js → GET /items → Worker → D1
```

Projektstruktur

```
spacesettlement/
├── assets/                Frontend Assets
│   └── js/
│       ├── app.js        Filter & Rendering
│       └── editor.js     Admin / Editor Logik
├── pages/                Statische Unterseiten
└── data/                 Konfigurationsdaten
```

```

├─ workers/
│   └─ spacesettlement-api/
│       ├── src/           Worker-Quellcode
│       ├── migrations/    D1 Migrationen
│       ├── public/        Worker Assets
│       ├── package.json
│       └─ wrangler.jsonc
├─ admin.html              Admin UI
├─ library.html            Index UI
├─ index.html              Landing
├─ .gitignore
├─ package.json
└─ README.md

```

Was gehört nicht ins Git-Repository

```

node_modules/
workers/**/*.node_modules/
workers/**/*.wrangler/
workers/**/*.dev.vars

```

Diese Verzeichnisse werden lokal erzeugt und müssen in `.gitignore` stehen.

Item-Datenmodell (D1: items)

Pflichtfelder: - `id` (TEXT, PK) - `type` (project | fiction | topic | org | person | book | movie | concept)
 - `title` - `createdAt`

Optionale Felder: - `summary` - `href` - `imageUrl` - `tags` - `startYear`, `endYear`, `sortYear` -
`project_class` - `fiction_class` - `budgetBillionUSD` - `meta`

Filterlogik (Frontend)

- Oberkategorien: `type` → projects, orgs, fiction, ...
- Unterkategorien:
- projects → `project_class`
- fiction → `fiction_class`
- topics → `meta.topicGroup`

Normalisierung erfolgt in `assets/js/app.js`.

Entwicklung

Frontend

```
npx serve .  
# oder Live Server in VS Code
```

Worker API

```
cd workers/spacesettlement-api  
npm install  
wrangler dev  
wrangler deploy
```

Ziel des Projekts

- Strukturierte, wartbare Wissensdatenbank
- Trennung von Inhalt (D1), Logik (Worker) und Darstellung (Frontend)
- Minimaler JS-Stack ohne Frameworks